TASK 03 – Prompting for Task Automation

Objective

The objective of this task was to design a reusable prompt capable of automating the conversion of unstructured meeting notes into a professional meeting report. The prompt was designed to consistently extract key information, organize action items, identify risks, and generate structured documentation suitable for project management workflows.

Unlike the previous tasks, this task was explored using **Google Gemini** to evaluate prompt performance on a different Large Language Model (LLM) and understand how prompt engineering principles transfer across AI platforms.

Introduction

Meeting documentation is a repetitive but essential activity in project management. Manual documentation can be time-consuming and inconsistent, especially when handling lengthy meeting discussions.

Through prompt engineering, Large Language Models can automate this process by extracting important information and presenting it in a standardized business format. This task focused on creating a reusable prompt that transforms raw meeting notes into professional meeting reports while maintaining accuracy and avoiding unsupported assumptions.

Prompt Used

Act as an experienced AI Project Management Assistant with expertise in project documentation *and meeting analysis.*

Your task is to convert the following meeting notes into a clear, professional, and structured meeting report.

Instructions:

- Extract only the important information.

- Remove duplicate or irrelevant details.

- Rewrite content in a concise and professional tone.
- Infer logical action items only when they are clearly implied; otherwise, mark missing information as "Not Specified."
- Preserve technical terms and project names.
- Do not invent deadlines, owners, or decisions.

Output Format:

 # Meeting Report

1. Meeting Summary
- Write a concise summary (3–5 sentences) covering:
  - Purpose of the meeting
  - Main discussion points
  - Overall outcome

2. Key Decisions
Provide a bullet list of all confirmed decisions.

3. Action Items

Task	*Owner*	*Priority*	*Deadline*	*Status*

Rules:
- Priority: High / Medium / Low
- If owner is missing, write "Not Specified."
- If deadline is missing, write "Not Specified."
- Default Status: Open

4. Risks or Blockers

List:

- Current risks

- Dependencies

- Issues delaying progress

- Open questions

If none are mentioned, write:

"No major risks or blockers identified."

5. Next Meeting Agenda

Generate a short agenda including:

- Review previous action items

- Discuss unresolved blockers

- Progress updates

- Upcoming milestones

- New discussion items (if applicable)

Formatting Requirements:

- Use professional business language.

- Keep the report concise and easy to scan.

- Use bullet points where appropriate.

- Present action items in a clean Markdown table.

- Do not include information that is not supported by the meeting notes.

Sample Input

Sample Meeting Notes

Project: Smart Inventory Management System

Date: 4 July 2026

Duration: 10:00 AM – 10:50 AM

Attendees: Priya (Project Manager), Rahul (Backend Developer), Neha (Frontend Developer), Arjun (ML Engineer), Kavya (QA Engineer)

Discussion Points

Priya reviewed the progress from the previous sprint.

Backend API development is approximately 80% complete.

Rahul mentioned that authentication APIs are finished, but inventory update APIs still need testing.

Neha completed the dashboard UI and started integrating backend APIs.

Some UI components are displaying incorrect stock values due to incomplete API responses.

Arjun demonstrated the demand prediction model with an initial accuracy of 89%.

The team agreed that additional historical sales data is needed to improve prediction accuracy.

Kavya reported that automated test cases cover around 65% of the application.

Login and registration modules passed all functional tests.

Inventory synchronization occasionally fails when the server experiences high traffic.

The deployment originally planned for Friday has been postponed because integration testing is not yet complete.

The client requested an export feature for inventory reports in Excel and PDF formats.

Rahul estimated that the export API would require approximately two additional development days.

Neha suggested adding a dark mode option, but the team decided to postpone it to the next release.

Arjun requested access to the latest sales database to retrain the prediction model.

Priya assigned Rahul and Neha to complete API integration before the next review meeting.

Kavya will begin regression testing once the API integration is completed.

The team discussed improving application performance by introducing Redis caching in a future sprint.

Everyone agreed to prioritize critical functionality over UI enhancements for the current release.

Decisions Made

Proceed with API integration before adding new features.

Postpone dark mode to the next release.

Delay deployment until integration testing is successfully completed.

Include the inventory export feature in the current sprint if time permits.

Action Items

Rahul: Complete inventory update APIs and export API.

Neha: Finish frontend-backend integration.

Arjun: Retrain the prediction model using updated sales data.

Kavya: Prepare regression test cases for integrated modules.

Priya: Coordinate with the client regarding the revised deployment schedule.

Risks / Issues

Delayed API integration may impact the deployment timeline.

Missing historical sales data could reduce AI model accuracy.

Server performance issues under heavy load require further investigation.

Export feature may extend the current sprint if additional changes are requested.

Next Meeting

Date: 8 July 2026

Agenda:

Review API integration progress.

Demonstrate the updated AI prediction model.

Review regression testing results.

Confirm deployment readiness.

Finalize sprint deliverables.

AI-Generated Output

Meeting Report

1. Meeting Summary

The primary purpose of the meeting was to review sprint progress for the Smart Inventory Management System. The team discussed ongoing backend and frontend API integration, evaluated the AI demand prediction model's accuracy, and addressed a new client request for report exports. Ultimately, the team decided to prioritize critical API integration over new UI enhancements and postponed the upcoming deployment until integration and regression testing are successfully completed.

2. Key Decisions

- **Prioritize Core Functionality:** Proceed with frontend-backend API integration before adding any new features.
- **Feature Deferment:** Postpone the suggested dark mode UI enhancement to the next release.
- **Schedule Adjustment:** Delay the project deployment until full integration testing is successfully completed.
- **Scope Addition:** Include the client-requested inventory report export feature (Excel/PDF) in the current sprint, pending available time.

3. Action Items

Task	Owner	Priority	Deadline	Status
Complete inventory update APIs and report export API	Rahul	High	Before next review	Open
Finish frontend-backend API integration	Neha	High	Before next review	Open
Retrain demand prediction model using updated sales data	Arjun	Medium	Not Specified	Open
Prepare regression test cases for integrated modules	Kavya	High	Not Specified	Open
Coordinate with the client regarding revised deployment schedule	Priya	High	Not Specified	Open

4. Risks or Blockers

- **Timeline Risk:** Delayed API integration and ongoing testing are impacting the overall deployment schedule.
- **Data Dependency:** Missing access to historical sales data is a blocker for improving the AI prediction model's accuracy.
- **Performance Issue:** Inventory synchronization occasionally fails under high server traffic, requiring further investigation (Redis caching planned for future).
- **Scope Creep:** The addition of the new report export feature may extend the duration of the current sprint.

5. Next Meeting Agenda

Date: 8 July 2026

- Review previous action items and discuss any unresolved blockers.
- Review progress on frontend-backend API integration.
- Demonstrate the updated AI demand prediction model.
- Review QA regression testing results.
- Confirm deployment readiness and finalize sprint deliverables.

Prompt Engineering Techniques Used

Technique	Purpose	Impact
Role Prompting	Assigned the AI the role of an experienced Project Management Assistant	Produced business-oriented responses
Structured Prompting	Defined a fixed report format with sections and tables	Generated consistent and organized outputs
Output Constraints	Prevented the AI from inventing deadlines, owners, or decisions	Improved reliability and factual accuracy
Context Engineering	Provided clear instructions for extracting only relevant information	Reduced unnecessary or duplicated content
Task Automation	Automated meeting documentation into a reusable workflow	Saved time and improved productivity

Output Analysis

The generated meeting report successfully converted lengthy meeting notes into a structured document containing:

- A concise meeting summary
- Confirmed project decisions
- Clearly organized action items
- Identified risks and blockers
- A suggested agenda for the next meeting

The AI preserved important technical terminology and project-specific information while removing redundant details. The Markdown table improved readability by organizing responsibilities, priorities, deadlines, and task status in a structured format suitable for project documentation.

Comparison

Aspect	Raw Meeting Notes	AI-Generated Report
Structure	Unorganized	Professional and structured
Readability	Moderate	Excellent
Action Items	Mixed within paragraphs	Organized in a table
Key Decisions	Embedded in discussion	Clearly separated
Risks	Difficult to identify	Highlighted explicitly
Meeting Summary	Not available	Automatically generated

Key Learnings

- Role prompting significantly improved the professionalism of the generated report.
- Structured prompts produced consistent outputs that could be reused across multiple meetings.
- Explicit constraints prevented the AI from generating unsupported information, improving trustworthiness.
- Prompt engineering can automate repetitive project documentation while maintaining clarity and accuracy.
- Testing the prompt with **Google Gemini** demonstrated that well-designed prompts remain effective across different LLMs, although response formatting and phrasing may vary slightly.

Reflection

Initially, a simple summarization prompt produced inconsistent results and often omitted important project information. By progressively refining the prompt with role assignment, detailed formatting instructions, output constraints, and validation rules, the generated reports became significantly more structured, reliable, and suitable for professional project management.

Using **Google Gemini** instead of ChatGPT for this task also provided insight into how prompt engineering techniques can be applied across different AI models. While the wording and presentation differed slightly, the core prompt design principles remained effective.

Conclusion

This task demonstrated the practical application of prompt engineering in automating project documentation. By transforming unstructured meeting notes into professional meeting reports, the prompt reduced manual effort, improved consistency, and produced outputs suitable for real-world software development and project management workflows. Additionally, experimenting with **Google Gemini** highlighted the adaptability of prompt engineering techniques across multiple Large Language Models.

Tanushri Mahesh Palleda,

Prompt Engineering Intern,

SkillCraft Technology.